Algorithm

Algorithm → It is step by step process of solving a Computational problem.

* Program is also step by step process of solving a computational problem.

See differences between Algorithm and program

Algorithm	Program
written at Design time	written at implementation time
Domain Knowledge	Programmer
Any language	only using programming language
H/W → Hardware OS → Operating system H/W & OS independent	dependent on H/W & OS
Analyze	Testing

Priori Analysis	Posteriori Testing
1. Algorithm	1. Program
2. Independent OF language	2. Language dependent
3. Hardware independent	3. Hardware dependent
4. Time & space Function	4. Watch time & Bytes

Characteristics OF Algorithm

1. Input → 0 or more input
2. Output → at least 1 O/P (√-1 not clear)
3. DeFiniteness → it should be clear (every statement)
4. Finiteness → must terminate at some point
5. EFFectiveness → Don't do unnecessary thing (means doing thing but OF no use)

How to write an Algorithm

```
Algorithm swap (a, b)
{
    temp = a;
    a = b;
    b = temp;
}
```

OR

```
Algorithm swap(a,b)
begin
    temp = a;
    a = b;
    b = temp
end
```

or

```
Algorithm swap (a, b)
Begin
    temp ← a;
    a ← b;
    b ← temp;
end
```

* You can use any syntax, you are free to use any syntax

How to Analyze an algorithm

1. Time
2. Space
3. N/w Consumption
4. Power Consumption
5. CPU Registers Consumption

* I don't know whether it is integer float or double that's why we have used word

Algorithm swap (a, b)

{

	Time	space
temp = a;	→ 1	a – 1
a = b;	→ 1	b – 1
b = temp;	→ 1	temp – 1
	f(n) = 3	s(n) = 3 word
	O(1)	O(1)

}

* every statement take One unit OF time

** statement can be complex at any level

(

ex:- x = 5*a + 6*b → 1

↓

IF we want to do detail analysis then above statement takes 4 unit OF time, generally we do brief analysis

Frequency Count method

Algorithm Sum (A, n)

A: | 8 | 5 | 9 | 7 | 2 |

0 1 2 3 4

5

{

s = 0; → 1

For (i = 0; i < n; i++)

{

s = s + A[i];

}

return s;

}

* Rule :- Assign one unit of time for each statement, If any statement is repeating for some number of time, the Frequency of execution of that statement will be calculated

```
Algorithm Sum (A, n)
{
    s = 0;                          ---------> 1
    For (i=0; i<n; i++)             ---------> n+1   [in delete it is 2n+1]
    {
        s = s + A[i];               ---------> n
    }
    return s;                       ---------> 1
}
```

i=0
i=1
i=2
i=3
i=4
i=5X

$F(n) = 2n + 3$

$O(n)$

Space

A → n
n → 1
s → 1
i → 1

$S(n) = n + 3$

$O(n)$

Sum of two matrics

```
Algorithm Add(A, B, n)
{
    For (i=0; i<n; i++)
    {
        For (j=0; j<n; j++)
        {
            C[i,j] = A[i,j] + B[i,j];
        }
    }
}
```

```
Algorithm Add (A, B, n)
{
    For (i=0; i<n; i++)  ⟶ n+1
    {
        For (j=0; j<n; j++)  → n*(n+1)
        {
            C[i,j] = A[i,j] + B[i,j];  → n*n
        }
    }
}
```

$f(n) = 2n^2 + 2n + 1$

$O(n^2)$

n×n

3×3

* whatever is inside this For loop will execute n times

```
For(i=0; i<n; i++) → n+1
{
    For(j=0; j<n; j++) → n
    {
        C[i,j] = A[i,j] + B[i,j]  → n
    }
}
```

explanation by breaking the step

```
For (i=0; i<n; i++) → n+1
{
    * whatever is inside this For loop will executes For n times
    For (j=0; j<n; j++) → n*(n+1)
    {
        C[i,j] = A[i,j] + B[i,j]   n*n
    }
}
```

Space

A → n^2

B → n^2

C → n^2

i → 1

j → 1

n → 1

$O(n^2)$.

$S(n) = 3n^2 + 3$

Q. Algorithm Multiply (A, B, n)

```
{
    For(i=0 ; i<n ; i++)
    {
        For(i=0 ; i<n ; i++
        {
            C[i,i] = 0;
        For(k=0; k<n ; k++)
        {
            C[i,j] = C[i,j] + A[i,k] * B[k,j]
        }
    }
}
```

Algorithm Multiply (A, B, n)

{

n+1 ← For (i=0; i<n; i++)

{

(n+1) n ← For (i=0; i<n; i++)

{

n x n ← C[i,j] = 0;

(n+1) x n ← For (k=0; k<n; k++)

{

n x n x n ← C[i,j] = C[i,j] + A[i,k] * B[k,j]

}

}

}

$F(n) = 2n^3 + 3n^2 + 2n + 1$

{

$O(n^3)$

Space

A → n^2

B → n^2

C → n^2

n → 1

i → 1

j → 1

k → 1

$S(n) = 3n^2 + 1$ → $O(n^2)$

Note-

You c

able t

direct

Comp

Tra

① a. For(i=0; i<n; i++) → n+1

{

stmt; ——— (n) → O(n)

}

① b. For(i=n; i>0; i--) → n+1

{

stmt; ——— (n) → O(n)

}

② a. For(i=1; i<n; i=i+2)

{

stmt; ——— (n/2) → O(n)

}

$F(n) = n/2$

② b. For(i=1; i<n; i=i+20)

{

stmt; ——— (n/20) → O(n)

}

$F(n) = n/20$

** In following questions directly we will not say it will execute for n times, we have to analyse the time

③ For(i=0; i<n; i++) → n+1

{

For(j=0; j<n; j++) → n×(n+1)

{

stmt; ——— (n×n) → $O(n^2)$

}

}

For(i=0; i<n; i++)

↓ n+1

For(j=0; j<i; j++)

→ n

{

stmt; → n

}

Note - whenever you are not able to find directly Time Complexity just Trace it & Find

For(i=0; i<n; i++)

{

For(j=0; j<i; j++)

{

stmt; ———

}

}

$1+2+3+\dots+n = \frac{n(n+1)}{2}$

$F(n) = \frac{n^2+1}{2}$ $O(n^2)$

i	j	no of times
0	0 x	0
1	0 ✓	1
	1 x	
2	0	2
	1	
	2 x	
3	0 ⎤	3
	1 ⎥	
	2 ⎥	
	3 x ⎦	
:	:	:
n		n

④ P = 0 ;

For (i=1; P<=n; i++)
{
P = P + i ;
}

i	P
1	0 + 1 = 1
2	1 + 2 = 3
3	1 + 2 + 3
4	1 + 2 + 3 + 4
.	
.	
.	
K	1 + 2 + 3 + 4 + ... + K

Assume P > n

$\therefore P = \frac{K(K+1)}{2}$

$\frac{K(K+1)}{2} > n$ (about)

$K^2 > n$

$K > \sqrt{n}$

$O(\sqrt{n})$

⑤ For (i=1; i<n; i = i*2)
{
stmnt ; → $\log_2 n$
}

i
1 x 2 = 2
2 x 2 = 2^2
2^2 x 2 = 2^3
.
.
2^K

$i = 1 \times 2 \times 2 \times 2 \cdots = n$

$2^K = n$

Assume $i >= n$

$\therefore i = 2^K$

$\therefore 2^K >= n$

$2^K = n$

$K = \log_2 n$ → $O(\log_2 n)$

** For(i=1 ; i<n ; i = i*2)

{

stmt ; ——— log n

}

observe it

n = 8 n = 10

i	
1	
2	3
4	
8 X	

i	
1	
2	4
4	
8	
16X	

$\log 8 = 3$

$\log_2 2^3 = 3 \log_2 2 = 3$

$\log_2 10 = 3.2$

$\therefore \lceil \log n \rceil$

⑥ For(i=n ; i>=1 ; i=i/2)

{

stmt ;

}

Assume $i < 1$

$\therefore \frac{n}{2^k} < 1$

$\frac{n}{2^k} = 1$

$\therefore k = \log_2 n \rightarrow O(\log_2 n)$

i
n
$n/2$
$n/2^2$
$n/2^3$
:
$n/2^k$

(7)

```
For (i=0; i*i<n; i++)
{
    stmnt;
}
```

$i*i<n$

$i*i>=n$

$i^2=n$

$i=\sqrt{n}$

(8)

```
For (i=0; i<n; i++)
{
    stmt;
}
```

—— n

```
For (j=0; j<n; j++)
{
    stmt;
}
```

→ n

$2n$ → $O(n)$

(9)

```
P=0
For (i=1; i<n; i=i*2)
{
    P++;
}
```

→ $\log n$

```
For (j=1; j<P; j=j*2)
{
    stmt;
}
```

—— $\log_2 P$

$P = \log n$

$O(\log \log n)$

(10)

```
For(i=0; i<n; i++)  ⟶ n
{
    For(j=1; j<n; j=j*2) - n × log n
    {
        stmt;  ⟶ n × log n
    }
}
```

$2n \log n + n$

$\downarrow$

$O(n \log n)$

For (i=0; i<n; i++) — $O(n)$

For (i=0; i<n; i=i+2) — $n/2$ $O(n)$

For (i=n; i>1; i--) — $O(n)$

For (i=1; i<n; i=i*2) — $O(\log_2 n)$

For (i=1; i<n; i=i*3) — $O(\log_3 n)$

For (i=n; i>1; i=i/2) ⟶ $O(\log_2 n)$

Analysis OF IF and while

```
i = 0;          ⟶ 1

while (i<n)     ⟶ n+1
{
    stmt;       ⟶ n
    i++;        ⟶ n
}
```

$f(n) = 3n+2$

$O(n)$

```
a = 1;
while (a<b)
{   stmt;
    a = a*2;
}
```

a
1
$*2 = 2$
$2 \times 2 = 2^2$
$2^2 \times 2 = 2^3$
⋮
2^k

Terminate → $2^k \geq b$

$k = \log_2 b$

```
i = n;
while (i>1)
{   stmt;       ⟶ log n
                   2
    i = i/2;
}
```

Note
are
Tim
dif

O(1)

```
i = 1;
k = 1;
while (k < n)
{
    stmt;
    k = k + i;
    i++;
}
```

i	k
1	1
2	1+1 = 2
3	2+2 =
4	2+2+3
5	2+2+3+4
:	:
:	:
:	:
n	2+2+3+4+ ---+n

$\frac{n(n+1)}{2}$ → roughly $=$

$k \geq n$

$\frac{n(n+1)}{2} \geq n$

$n^2 \geq n$

$n = \sqrt{n}$ → $O(\sqrt{n})$

$=$

```
while (n != n)
{
    IF (n > n)
        n = n - n;
    else
        n = n - n;
}
```

n	n	
6	3	→ 1
3	3	
n	n	→ 0
5	5	

Note - whenever you are not able to find Time complexity directly, just trace it

n	n	
16	2	
14	2	
12	2	→ 7
10	2	
8	2	
6	2	
4	2	
2	2	

$O(1)$ ← Min

Max → $O(n)$

```
Algorithm Test(n)
{
    IF (n<5)
    {
        PrintF("%d", n);  --> 1
    }
    else
    {
        For(i=0; i<n; i++)
        {
            Printf("%d", i);  --> n
        }
    }
}
```

Best → O(1)

worst → O(n)

Types OF Time Functions / classes OF Function

O(1) → Constant class

O(logn) → logarithmic class

O(n) → Linear

$O(n^2)$ → Quadratic

$O(n^3)$ → Cubic

$O(n^n)$ / $O(n!)$ / $O(2^n)$ → Exponential

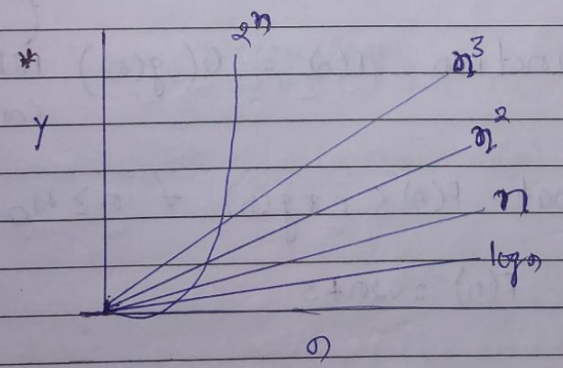
$\log n < \sqrt{n} < n < n \log n < n^2 < n^5 \; \cdots \; < 2^n < 8^n \cdots < n^n$

$\log n$	n	n^2	2^n
0	1	1	2
$\log_2^2 = 1$	2	4	4
2	4	16	16
3	8	64	256
3.1	9	81	512

→ For smaller value of n, 2^n may be lesser or equal to n [~~similar~~ For other ~~also~~] but For larger value 2^n is surely bigger

$n^{100} < 2^n$

$n^K < 2^n$



→ initially value of 2^n is less after Then there is sudden increase

g.log8

Asymptotic Notation

$1 < \log n < \sqrt{n} < n < n\log n < n^2 < n^3 < \dots < 2^n < 8^n < \dots < n^n$

O big-oh upper bound

Ω big-omega Lower bound

useful → Θ theta Average bound

Big-oh

The Function $F(n) = O(g(n))$ IFF ∃ +ve constant c & n_0 (IF & only IF there exist)

such that $F(n) \le c * g(n)$ $\forall$ $n \ge n_0$

eg $F(n) = 2n+3$

$2n+3 \le 10n$ $\quad n \ge 1$

($2n+3$ = $F(n)$, 10 = c, n = $g(n)$)

$\therefore F(n) = O(n)$

↓ * technique to find

$2n+3 \le 2n+3n$

$2n+3 \le 5n$

$n \ge 1$

No

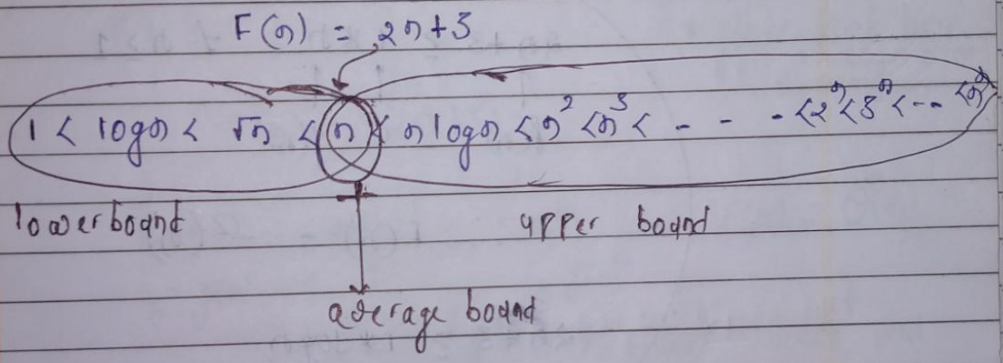
$F(n) = 2n+3$

$2n+3 \leq 2n^2 + 3n^2$

$2n+3 \leq 5(n^2) \quad n \geq 1$

$F(n) \quad \in g(n)$

$F(n) = O(n^2)$ ~ this is also true



Note:

✓ $F(n) = O(n)$

✓ $F(n) = O(n^2)$

✓ $F(n) = O(2^n)$

↳ All these coming on right side

✗ $F(n) = O(\log n)$

↓

coming left side

Note:- When you write big-oh try to write closest Function, In above case it is n. If you write bigger Function it is also true. though not useful.

Omega

The Function $F(n) = \Omega(g(n))$ IFF $\exists$ +ve constant c and n_0

such that $F(n) \geq c * g(n)$ $\forall$ $n \geq n_0$

e.g: $F(n) = 2n+3$

$2n+3 \geq 1 * n$ $\forall$ $n \geq 1$

$f(n)$ c $g(n)$

$\therefore F(n) = \Omega(n)$

$2n+3 \geq 1 * \log n$

$\therefore F(n) = \Omega(\log n)$

} this also true

$1 < \log n < \sqrt{n} < (n) < n \log n < n^2 < n^3 < \cdots < 2^n < 3^n \cdots n^n$

lower Found e.g. bound upper bound

✓ $F(n) = \Omega(n)$

✓ $F(n) = \Omega(\log n)$

✗ $F(n) = \Omega(n^2)$

Note:- Try to write nearest one

Theta Notation

The Function $F(n) = \theta(g(n))$ IFF $\exists$ +ve constant C_1, C_2 and n_0

such that $C_1 * g(n) \leq F(n) \leq C_2 * g(n)$

* Note that here $g(n)$ at both side is same $\leq$

e.g: $F(n) = 2n+3$

$1 \times n \leq 2n+3 \leq 5 \times n$

$C_1 \times g(n)$ $\quad F(n) \quad$ $C_2 g(n)$

$\therefore F(n) = \theta(n)$

* above thing shows that $F(n)$ is exactly equal to n

$1 < \log n < \sqrt{n} < (n) < n \log n < n^2 < n^3 < \cdots < 2^n < 5^n < \cdots < n^n$

e.g bound

✓ $F(n) = \theta(n)$

✗ $F(n) = \theta(n^2)$

$\rightarrow 1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 ----- < 2^n < 3^n -- < n^n$

e.g $F(n) = 2n^2 + 3n + 4$

$2n^2 + 3n + 4 \leq 2n^2 + 3n^2 + 4n^2$

$2n^2 + 3n + 4 \leq g(n^2)$

$c \quad g(n)$

$F(n) = O(n^2)$

$2n^2 + 3n + 4 \geq 1 \times n^2$

$F(n) = \Omega(n^2)$

$1 \times n^2 \leq 2n^2 + 3n + 4 \leq g(n^2)$

$F(n) = \Theta(n^2)$

e.g: $F(n) = n^2 \log n + n$

$\frac{n^2 \log n}{1} \leq n^2 \log n + n \leq \frac{10 n^2 \log n}{1}$

$O(n^2 \log n)$, $\Omega(n^2 \log n)$

$\Theta(n^2 \log n)$

Eg:- $F(n) = n!$

$n! = n(n-1) \cdots \times 3 \times 2 \times 1$

$1 \times 1 \times 1 \cdots \times 1 \leq 1 \times 2 \times 3 \times \cdots \times n \leq n \times n \times n \cdots \times n$

$1 \leq n! \leq n^n$

$\Omega(1)$ $O(n^n)$

Here we can't Find average bound

Eg:- $F(n) = \log n!$

$\log(1 \times 2 \times 3 \times \cdots n)$

$\log(1 \times 1 \times 1 \times \cdots 1) \leq \log(1 \times 2 \times 3 \cdots \times n) \leq \log(n \times n \times \cdots n)$

$1 \leq \log n! \leq \log n^n$

$= n \log n$

$\Omega(1)$ $O(n \log n)$

Note:- Always Θ is preferrable. If you are able to Find Θ For any Func^n that is better

Properties of Asymptotic Notations

General properties

- IF $F(n)$ is $O(g(n))$ then $a * f(n)$ is $O(g(n))$

  eg:- $F(n) = 2n^2 + 5$ is $O(n^2)$

  then $7F(n) = 7(2n^2 + 5)$

  $= 14n^2 + 35$ is $O(n^2)$

- IF $F(n)$ is $\Omega(g(n))$ then $a * f(n)$ is $\Omega(g(n))$
- IF $F(n)$ is $\theta(g(n))$ then $a * F(n)$ is $\theta(g(n))$

Reflexive properties

IF $F(n)$ is given then $F(n)$ is $O(f(n))$

eg:- $F(n) = n^2$ $O(n^2)$

Similarly For Omega also; θ also

Transitive

IF $F(n)$ is $O(g(n))$ and $g(n)$ is $O(h(n))$

then $F(n) = O(h(n))$

eg:- $F(n) = n$ $g(n) = n^2$ $h(n) = n^3$

n is $O(n^2)$ and n^2 is $O(n^3)$

then n is $O(n^3)$

$g(n)$ is upper bound of $F(n)$ and $h(n)$ is upper bound of $g(n)$ then obviously $g(n)$ is upper bound of $F(n)$

* Transitive properties is valid for theta and Omega also.

Symmetric

IF $F(n)$ is $\theta(g(n))$ then $g(n)$ is $\theta(f(n))$

* it is true only for θ notation.

eg:- $F(n) = n^2$, $g(n) = n^2$

$F(n) = \theta(n^2)$

$g(n) = \theta(n^2)$

Transpose symmetric

* It is true only for O and ω notation

IF $F(n) = O(g(n))$ then $g(n)$ is $\Omega(f(n))$

eg:- $F(n) = n$ $g(n) = n^2$

then n is $O(n^2)$ and

n^2 is $\Omega(n)$

- IF $F(n) = O(g(n))$
  and $F(n) = \Omega(g(n))$
  →
  $g(n) \leq F(n) \leq g(n)$
  $\therefore F(n) = \theta(g(n))$

Q. IF $F(n) = O(g(n))$ and $d(n) = O(e(n))$
then $F(n) + d(n) = ?$

→
$F(n) = n \rightarrow O(n)$
$d(n) = n^2 \rightarrow O(n^2)$
$F(n) + d(n) = n + n^2 = O(n^2)$

Ans → $O(max(g(n), e(n)))$

Q. IF $F(n) = O(g(n))$
and $d(n) = O(e(n))$
then $F(n) * d(n) = ?$

→
$O(g(n) * e(n))$

Comparision of Functions

n^2 n^3

M1 → by putting values

$2^2 = 4$ $2^3 = 8$

$3^2 = 9$ $3^3 = 27$

$n^2 < n^3$

M2 → Apply log on both sides

$\log n^2$ $\log n^3$

$2\log n < 3\log n$

$\log ab = \log a + \log b$
$\log a/b = \log a - \log b$
$\log a^b = b\log a$
$a^{\log_c b} = b^{\log_c a}$
$a^b = n$ then $b = \log_a n$

eg → $F(n) = n^2 \log n$, $g(n) = n(\log n)^{10}$

Apply log

$\log[n^2 \log n]$ $\log[n(\log n)^{10}]$

$\log n^2 + \log\log n$ $\log n + \log(\log n)^{10}$

$2\log n + \log\log n$ > $\log n + 10\log\log n$

1. $F(n) = 3n^{\sqrt{n}}$ $\quad g(n) = 2^{\sqrt{n}\log n}$

→

$3n^{\sqrt{n}}$ $\quad 2^{\sqrt{n}\log n}$

↓

$3n^{\sqrt{n}}$ $\quad 2^{\log_2 n^{\sqrt{n}}} \rightarrow (n^{\sqrt{n}})^{\log_2 2}$

$3n^{\sqrt{n}} \rightarrow n^{\sqrt{n}}$

* Asymptotically above are equal (as both are $O(n^{\sqrt{n}})$, IF asked Value wise then First one is greater

2. $F(n) = n^{\log n}$ $\quad g(n) = 2^{\sqrt{n}}$

Apply log

$\log n^{\log n}$ $\quad \log 2^{\sqrt{n}}$

$\log n \times \log n$ $\quad \sqrt{n} \log_2 2^1$

$\log^2 n$ $\quad \sqrt{n}$

* If you are unable to judge it still you can apply log

$2 \log \log n$ $\quad < \quad \frac{1}{2} \log n$

3. $F(n) = 2^{\lg n}$ $\quad g(n) = n^{\sqrt{n}}$

→ $\log n \times \log_2^2$ $\quad \sqrt{n} \log n$

$\log n \quad < \sqrt{n} \log n$

4. $F(n) = 2n \quad g(n) = 5n$

→

$F(n) = g(n)$ [asymptotically]

5. $F(n) = 2^n \quad g(n) = 2^{2n}$

→

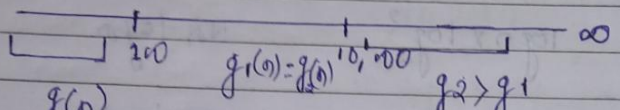
$\times \quad n \log 2 \quad = \quad 2n \log 2$

Don't say they are equal, we have applied log and then we are comparing [Directly in f^n you can ~~say that~~ ≠ ~~cutoff~~ coeff.]

$2^n < 2^{2n}$

6. $g_1(n) = \begin{cases} n^5 & n < 100 \\ n^2 & n \geq 100 \end{cases}$

$g_2(n) = \begin{cases} n^2 & n < 10,000 \\ n^3 & n \geq 10,000 \end{cases}$



$g_2 > g_1$

7. Find T / F

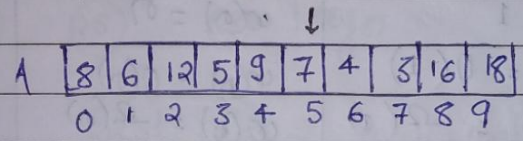
1. $(n+k)^m = \Theta(n^m)$ → T
2. $2^{n+1} = O(2^n)$ → T
3. $2^{2n} = O(2^n)$ → F ($4^n > 2^n$)
4. $\sqrt{\log n} = O(\log \log n)$ → F
5. $n^{\log n} = O(2^n)$ → T

i

Best, worst and Average case Analysis

1. Linear search
2. Binary search

1. Linear search



Key = 7

Best Case - If searching key element present at First index

Best Case Time - 1 $O(1)$

$B(n) = O(1)$

Worst Case - Searching key at last index

worst Case Time - n

$W(n) = n$

$O(n)$

Average Case - $\frac{\text{all possible case time}}{\text{no. of cases}}$

Avg time - $\frac{\frac{n(n+1)}{2}}{n} = \frac{n+1}{2}$

$A(n) = \frac{n+1}{2}$

$B(n) = O(1)$

$\omega(n) = O(n)$

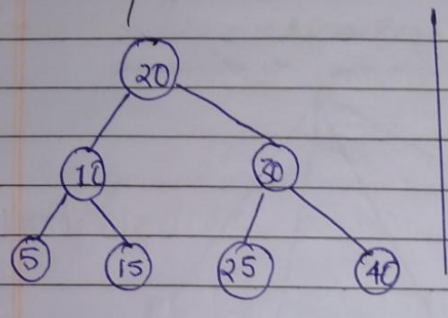
$A(n) = \frac{n+1}{2}$

Applying Notation

$B(n) = 1$	$\omega(n) = n$	$A(n) = \frac{n+1}{2}$
$B(n) = O(1)$	$\omega(n) = O(n)$	$A(n) = O(n)$
$B(n) = \Omega(1)$	$\omega(n) = \Omega(n)$	$A(n) = \Omega(n)$
$B(n) = \Theta(1)$	$\omega(n) = \Theta(n)$	$A(n) = \Theta(n)$

*** Don't take it wrong as Bigoh is for worst case, Ω is for best case → this perception/ thinking is wrong - you can use Θ also for best case & worst case O for best case & worst case Ω also for best case & worst case

Binary search tree

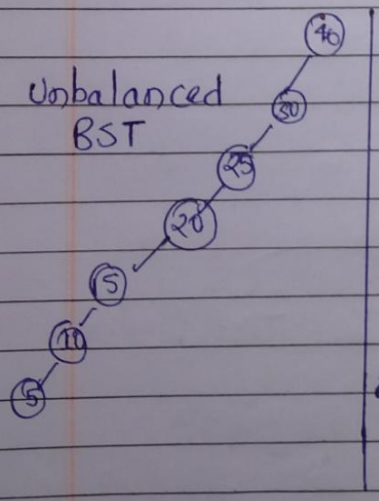


Best case - Search root element

Best case Time - $B(n) = 1$

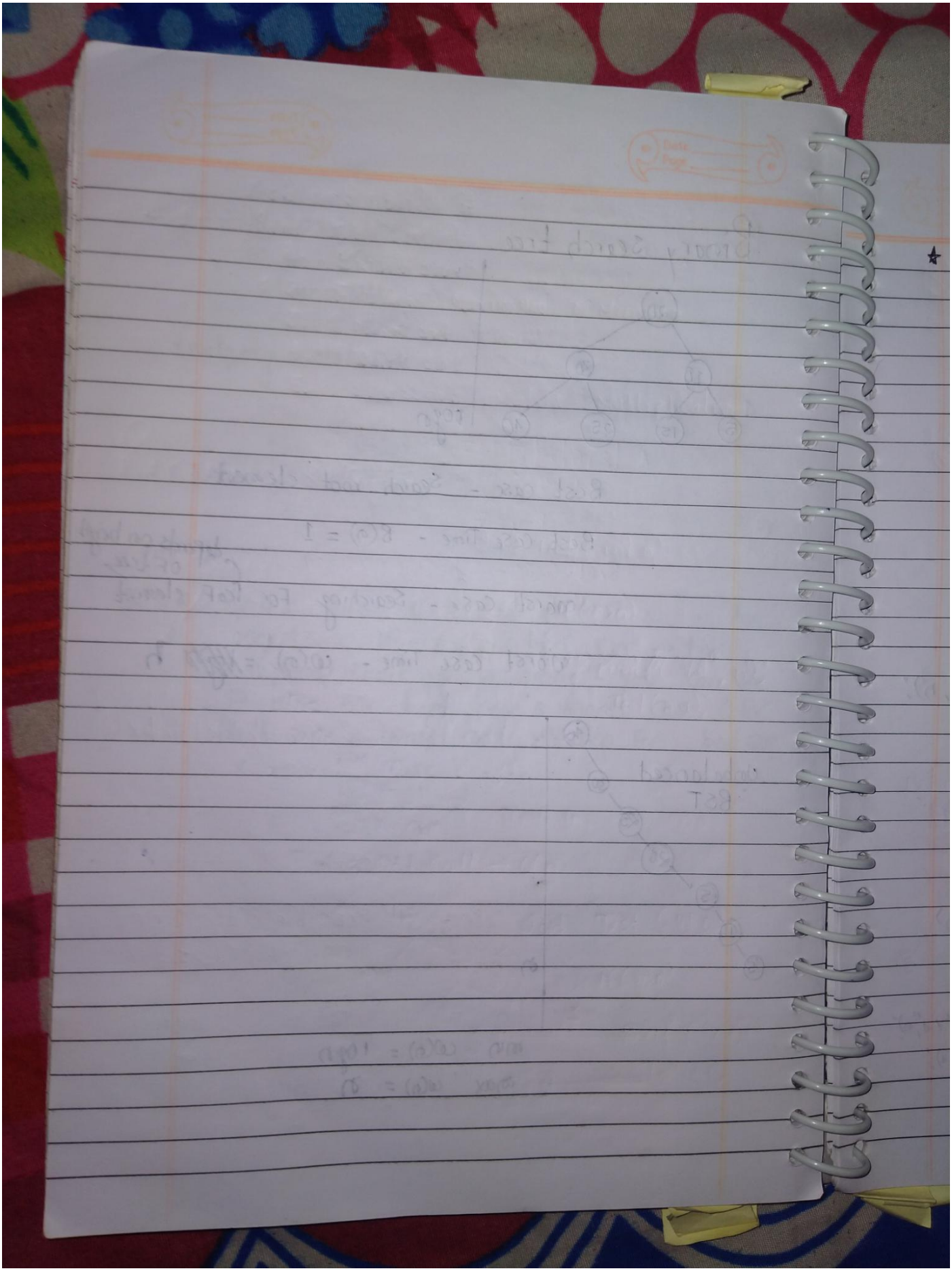
Worst case - Searching For leaf element (depends on height of tree)

Worst case Time - $W(n) = h$



min $W(n) = \log n$

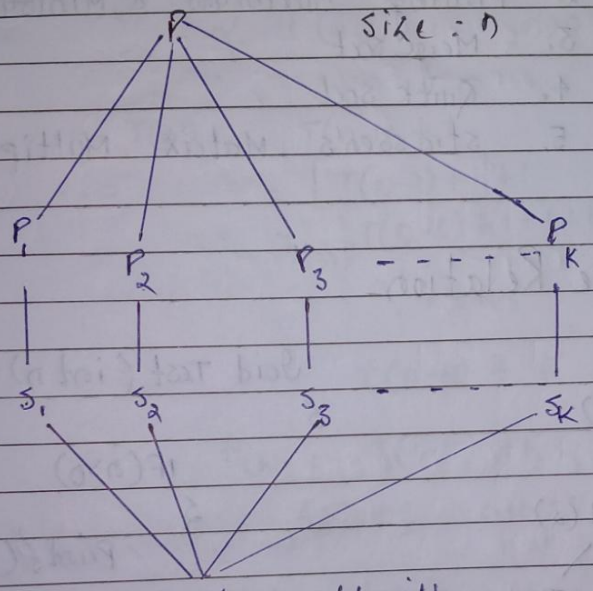
max $W(n) = n$



Divide and Conquer

★ It is one of the strategy for solving a problem.

*P → Problem *n is size of input



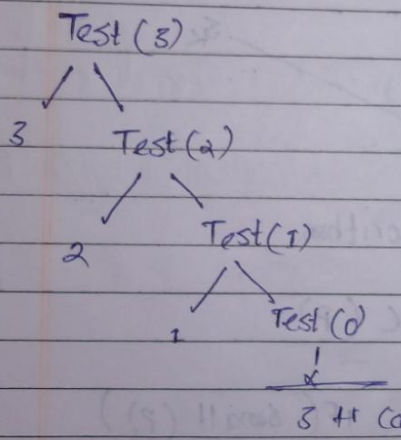
Algorithm

```
DAC (P)
{
    IF( small (P))
    {
        s(P);
    }
    else
    { divide P into P1, P2, P3 -- Pk
      Apply DAC(P1), DAC(P2), ---
      Combine (DAC(P1), DAC(P2), --)
    }
```

Problems under it

1. Binary search
2. Finding Maximum & Minimum
3. Merge sort
4. Quick sort
5. Strassen's Matrix Multiplication

Recurrence Relation




```
void Test (int n)
{
    IF(n>0)
    {
        Printf("%d", n);
        Test(n-1);
    }
}
```

$F(n) = n+1$ call $\rightarrow O(n)$

```
void Test (int n)
{
    IF(n>0)
    {
        Printf("%d", n);   1
        Test(n-1);          T(n-1)
    }
}
```

$T(n) = T(n-1) + 1$ → here at place of 1, you can use c_k also

$$T(n) = \begin{cases} 1 & n = 0 \\ T(n-1) + 1 & n > 0 \end{cases}$$

$T(n) = T(n-1) + 1$

$= [T(n-2) + 1] + 1$

$= T(n-3) + 1 + 1 + 1$

⋮ k times

$T(n-k) + k$

$T(n) = T(n-k) + k$

Assume $n - k = 0$

$\therefore n = k$

$T(n) = T(0) + n$

$T(n) = 1 + n$

$\Theta(n)$

```
T(n) ← 1.   void Test(int n)
            {
          1 ——  IF(n>0)
            {
          n+1 —— For(i=0; i<n; i++)
            {
          n ——      printf("%d", n);
            }
          T(n-1) —— Test(n-1);
            }
```

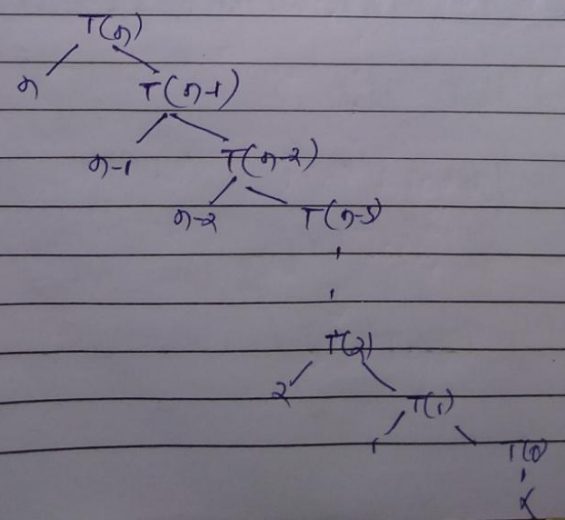
→

$T(n) = T(n-1) + (2n+2)$

for solving easily replace by

$T(n) = T(n-1) + n$

$$T(n) = \begin{cases} 1 & n=0 \\ T(n-1)+n & n>0 \end{cases}$$



$n + (n-1) + (n-2) + \cdots + 2 + 1$

$= \frac{n(n+1)}{2}$

$\Theta(n^2)$

Or

$T(n) = T(n-1) + n$

$T(n-2) + (n-1) + n$

$T(n-2) + (n-2) + (n-1) + n$

$\vdots$

$T(0) + 1 + 2 + 3 + \cdots n$

$\Theta(n^2)$

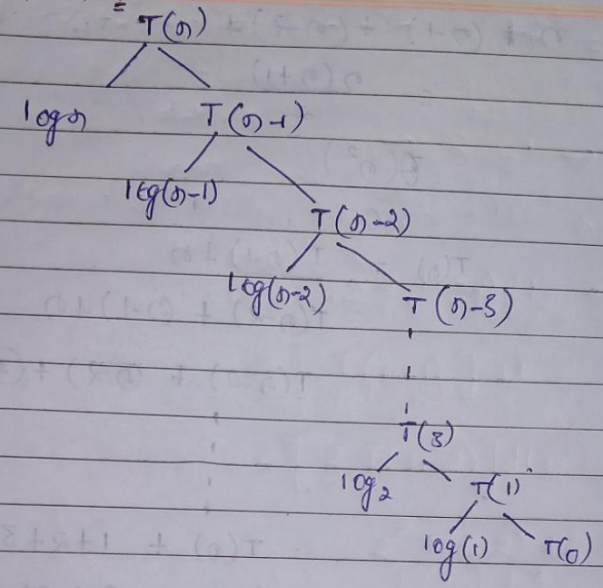
2.
```
void Test(int n)
{
    IF (n>0)
    {   For(i=1; i<n; i=i*2)
        {
            Printf("%d", i);
        }
        Test(n-1);
    }
}
```

$T(n-1)$

$T(n) = T(n-1) + \log n$

$T(n) = \begin{cases} 1 & n = 0 \\ T(n-1) + \log n & n > 0 \end{cases}$

By Tree Method



$$\log n + \log(n-1) + \dots + \log 2 + \log 1$$
$$= \log(n * (n-1) * \dots * 2 * 1)$$
$$= \log(n!)$$
$$\boxed{O(n \log n)}$$

M-2

$$T(n) = T(n-1) + \log n$$
$$T(n-2) + \log(n-1) + \log n$$
$$\vdots$$
$$T(0) + \log n!$$
$$T(0) = 1 + \log n!$$
$$\downarrow$$
$$O(n \log n)$$

* Short Trick For Finding recurrence relation

$T(n) = T(n-1) + 1$ — $O(n)$

$T(n) = T(n-1) + n$ — $O(n^2)$

$T(n) = T(n-1) + \log n$ — $O(n \log n)$

$T(n) = T(n-1) + n^2 = ?$ $O(n^3)$

$T(n) = T(n-2) + 1 = ?$ $n/2 \rightarrow O(n)$

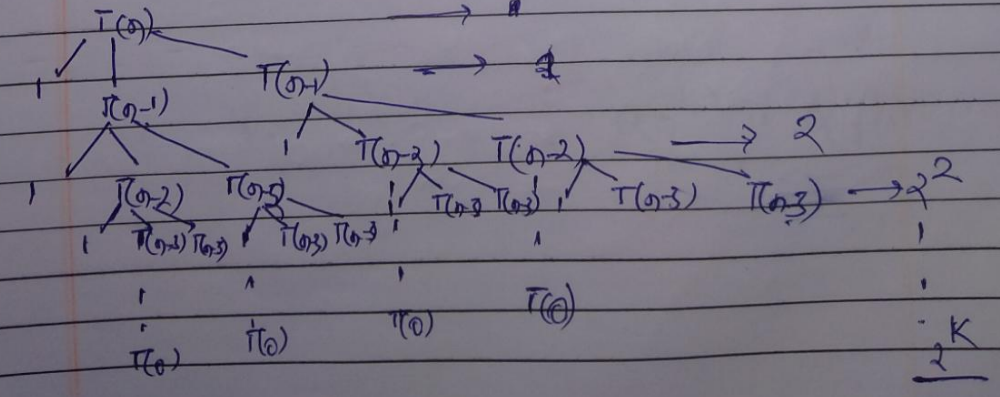
$T(n) = T(n-10) + n = ?$ $O(n^2)$

3. Algorithm Test (int n)

```
{
    IF(n>0)
    {
        printf("%d", n);
        Test(n-1);
        Test(n-1);
    }
}
```

→ $T(n) = 2T(n-1) + 1$

~~$2T(n-2) + 1 + 2 \cdot 2$~~



$1 + 2 + 2^2 + 2^3 + \cdots + 2^K = 2^{K+1} - 1$

$n - K = 0$

$K = n$

$\rightarrow 2^{n+1} - 1 \rightarrow O(2^n)$

$$T(n) = \begin{cases} 1 & n = 0 \\ 2\,T(n-1) + 1 & n > 0 \end{cases}$$

$T(n) = 2\,[\,T(n-1)\,] + 1$

$= 2\,[\,2\,T(n-2)\,] + 1$

$= 2^2\,T(n-2) + 2 + 1$

$= 2^2\,[\,2\,T(n-3) + 1\,] + 2 + 1$

$= 2^3\,T(n-3) + 2^2 + 2 + 1$

$T(0) = 0$

$\vdots$

$T(n) = 2^n + 2^{n-1} + 2^{n-2} + \cdots + 2^2 + 2 + 1$

$\hookrightarrow O(2^n)$

Master Theoroom For Decreasing Function

$T(n) = T(n-1) + 1 = O(n)$

$T(n) = T(n-1) + n = O(n^2)$

$T(n) = T(n-1) + \log n = O(n \log n)$

$T(n) = 2T(n-1) + 1 = O(2^n)$

$T(n) = 3T(n-1) + 1 = O(3^n)$

$T(n) = 2T(n-1) + n = O(n 2^n)$

$T(n) = aT(n-b) + F(n)$

$a > 0$, $b > 0$ and $F(n) = O(n^K)$ where $K \geq 0$

IF $a = 1 \rightarrow O(n^{K+1})$ or $O(n * f(n))$

IF $a > 1 \rightarrow O(n^K a^{n/b})$ or $O(f(n) * a^{n/b})$

$T(n) = 2T(n-2) + 1 = O(2^{n/2})$

IF $a < 1 \rightarrow O(n^K)$ or $O(f(n))$

2.5.1

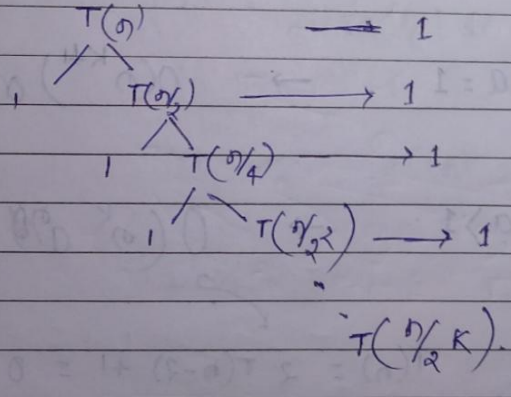
Dividing Functions

1. Algorithm Test (int n) — T(n)

```
{
    IF(n>1)  ——— 1
    {
        Printf("%d", n);
        Test(n/2);  ——→ T(n/2)
    }
}
```

$$T(n) = T(n/2) + 1$$

$$T(n) = \begin{cases} 1 & n=1 \\ T(n/2) + 1 & n>1 \end{cases}$$



Since $n/2^K = 1$

$K = \log_2 n \rightarrow O(\log n)$

$$T(n) = \begin{cases} 1 & n = 1 \\ T(n/2) + 1 & n > 1 \end{cases}$$

$$T(n) = T(n/2) + 1 \quad \text{---} (1)$$

$$T(n) = \left[T\left[\frac{n}{2^2}\right] + 1 \right] + 1$$

$$\vdots$$

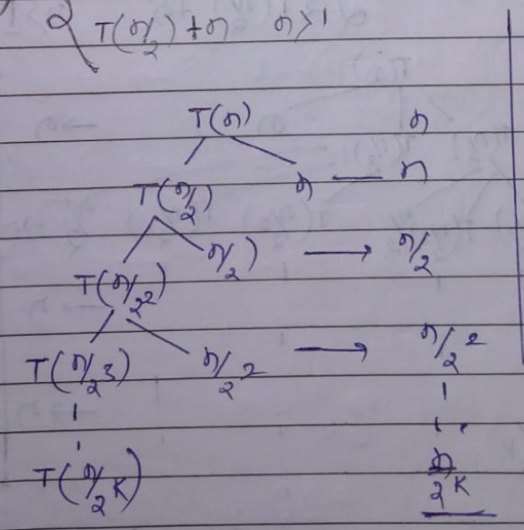
$$T(n) = T(n/2^k) + k$$

Assume $n/2^k = 1$

$$k = \log_2 n$$

$$O(\log n)$$

Q2. $$T(n) = \begin{cases} 1 & n = 1 \\ T(n/2) + n & n > 1 \end{cases}$$



$$T(n) = n + \frac{n}{2} + \frac{n}{2^2} + \frac{n}{2^3} + \cdots + n/2^k$$

$$= n\left[1 + 1/2 + 1/2^2 + 1/2^3 + \cdots + 1/2^k\right]$$

$$= n \left(\sum_{i=0}^{k} \frac{1}{2^i} \right) = 1$$

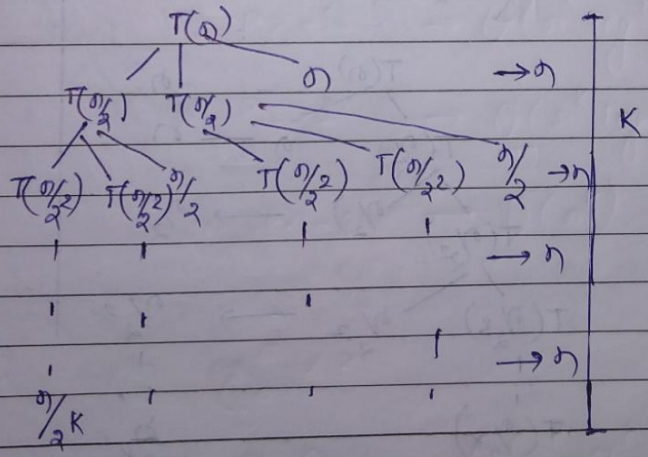
$$T(n) = n \rightarrow O(n)$$

3. $T(n)$ ← void Test(int n)

```
{
    IF(n>1)
    {
        For(i=0; i<n; i++)        n
        {
            stmnt;
        }
    T(n/2) ← Test(n/2);
    T(n/2) ← Test(n/2);
    }
}
```

$$T(n) = 2T(n/2) + n$$

$$T(n) = \begin{cases} 1 & n = 1 \\ 2T(n/2) + n & n > 1 \end{cases}$$



$nK \rightarrow O(n \log n)$

$T(n) = 2T(n/2) + n$ — (1)

$= 2\left[2T(n/2^2) + n/2\right] + n$

$= 2^2 T(n/2^2) + n + n$

$= 2^2\left[2T(n/2^3) + n/2^2\right] + 2n$

$= 2^3 T(n/2^3) + 3n$ — (2)

⋮

$T(n) = 2^k T(n/2^k) + kn$

$T(\frac{n}{2^k}) = T(1)$

$\therefore n/2^k = 1 \quad n = 2^k$

$k = \log n$

$O(n \log n)$

Master Theorem Algorithms For Divisions

Ex-

→

General Form

$$T(n) = a\,T(n/b) + F(n)$$

Assume

$a \geq 1$

$b > 1$

$F(n) = \Theta(n^K \log^P n)$

① $\log_b^a$

② K

Case 1: IF $\log_b^a > K$ then $\Theta(n^{\log_b^a})$

Case 2: IF $\log_b^a = K$

IF $P > -1$ → $\Theta(n^K \log^{P+1} n)$

IF $P = -1$ → $\Theta(n^K \log \log n)$

IF $P < -1$ → $\Theta(n^K)$

Case 3: IF $\log_b^a < K$

IF $P \geq 0$ $\Theta(n^K \log_n^P)$

IF $P < 0$ $O(n^K)$

Ex:- $T(n) = 2T(n/2) + 1$

→ $a = 2$

$b = 2$

$F(n) = \theta(1)$

$= \theta(n^0 \log^0 n)$

$K = 0 \quad P = 0$

$\log_2^2 = 1 > K = 0$

Case → 1 $\quad \theta(n^1)$